ILP-Based Scheduling for High-Level Synthesis

Milestone 3: Critical Evaluation and Transfer

Name	MD Rafiul Islam
Student ID	1232714
Course area	SS2026 ELE HW/SW Codesign Seminar
Topic	ILP-Based Scheduling for High-Level Synthesis
Document type	Critical evaluation report and transfer notes

Purpose: This document is a structured milestone artifact for understanding, analysis, evaluation, and scientific communication. It is written as technical notes, not as copied final prose.

1. Milestone 3 goal

Milestone 3 evaluates the approach critically. The focus is not whether ILP-based scheduling is simply good or bad. The focus is under which assumptions it is useful, where it may fail, and how it transfers to realistic embedded systems and RTL hardware.

2. Strengths of ILP-based scheduling

- Formal correctness of the schedule can be checked through explicit constraints.
- The objective is transparent: for example minimize latency under fixed resource limits.
- Resource tradeoffs are easy to demonstrate by changing the available number of functional units.
- The small example clearly shows that two multipliers reduce latency from 3 cycles to 2 cycles.
- The model is explainable: each schedule decision maps to a binary variable and each design rule maps to a constraint.
- It is useful for design-space exploration and for validating small critical kernels.

3. Limitations and risks

Issue	Why it matters	Mitigation or note
Scalability	Variables grow with operations times possible cycles.	Use solvers, time windows, decomposition, or heuristics.
Simplified operation delays	Real designs may use multi-cycle or pipelined units.	Extend constraints with operation-specific latencies.
Memory behavior	Memory ports and dependencies can dominate schedules.	Add memory/resource constraints and alias assumptions.
Branches and loops	Control flow creates multiple paths and iteration constraints.	Use control/data-flow graphs and loop scheduling techniques.
Manual RTL mapping	A schedule is not automatically a complete HLS tool.	Clearly present VHDL as realization/validation of schedule results.
Solver dependence	Professional ILP solvers may be needed for larger cases.	State the difference between pedagogical search and industrial solving.

4. Runtime and overhead evaluation

The computational cost is mainly determined by the number of binary variables and constraints. If there are n operations and T possible control steps, there are roughly n*T operation-to-cycle variables before additional auxiliary variables are considered.

```
Number of assignment variables: approximately |V| * T
Uniqueness constraints: approximately |V|
Dependency constraints: approximately |E|
Resource constraints: approximately |ResourceTypes| * T
```

This is why ILP is attractive for optimality but risky for large designs. The seminar C++ implementation avoids a solver and uses exhaustive search because the example has only three operations. This is useful pedagogically but should not be oversold as scalable industrial scheduling.

5. Transfer to realistic applications

Application fit	Example	Reason
Suitable	Small arithmetic accelerator kernel	The operation graph is compact and exact latency/resource tradeoff matters.
Suitable	FPGA DSP block exploration	Changing multiplier count maps naturally to hardware resource tradeoffs.
Suitable	Academic HLS explanation	The binary variable and constraint model is easy to inspect.
Less suitable	Large control-heavy software with branches	Model may become complicated and large.
Less suitable	Memory-dominated applications	Memory dependencies and port conflicts may dominate the schedule.
Less suitable	Fully dynamic runtime workloads	HLS scheduling is normally design-time/offline.

6. Possible extensions and open research questions

- Replace exhaustive search with a real ILP solver and compare runtime for larger graphs.
- Add multi-cycle operation delays for realistic multipliers and dividers.
- Add memory-port constraints and load/store dependencies.
- Add area-weighted or power-aware objectives instead of only latency minimization.
- Compare ILP schedules with list scheduling and force-directed scheduling on the same graph.
- Extend the RTL generation step from manually written schedules to automatically generated VHDL state machines.

7. Critical conclusion for the paper

The strongest claim should be modest: ILP-based scheduling gives a clear and exact formulation for small HLS scheduling problems and demonstrates the resource-latency tradeoff cleanly. It should not be claimed as universally practical for all industrial HLS designs without discussing scalability and model complexity.

Reference	How it supports this topic
Teich and Haubelt, 2007	German HLS / HW/SW synthesis background: allocation, scheduling, binding, optimization.
De Micheli, 1994	Main textbook-level foundation for synthesis and optimization of digital circuits.
McFarland, Parker, Camposano, 1990	Classic overview of high-level synthesis and behavioral synthesis flow.
Paulin and Knight, 1989	Neighboring heuristic approach: force-directed scheduling for behavioral synthesis.
Cong et al., 2011	Modern FPGA-oriented HLS context and deployment relevance.
Rettberg, 2026 seminar slides	Course requirements: scientific paper, C/C++ feature, application example, RTL component realization.